

Projet de Développement LU2IN013

Membres du groupe:

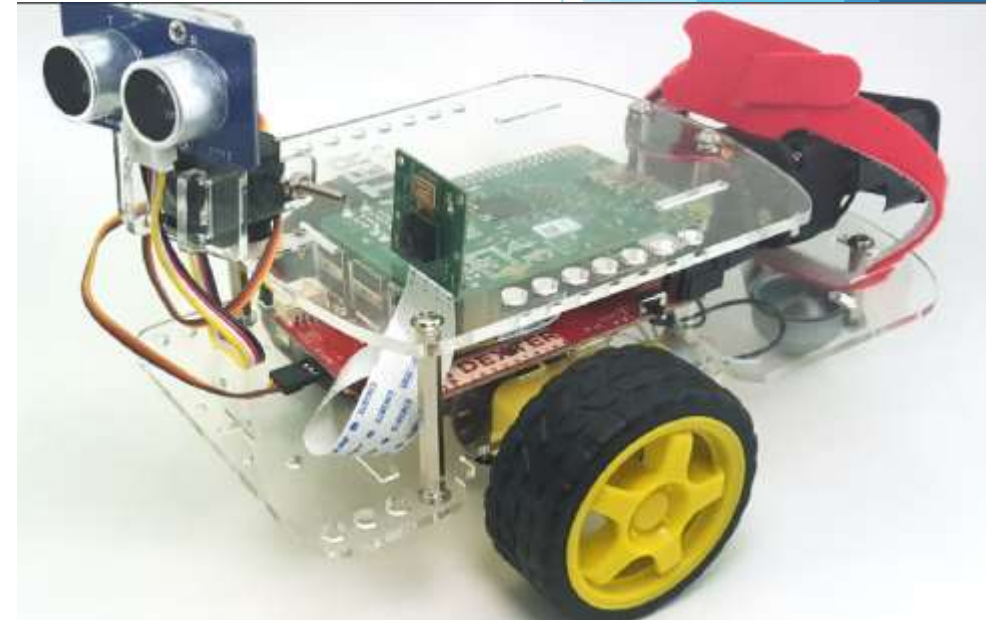
- AKSAS TASSADIT
- ARKAM WISSAM
- ARAOUR DIANA
- BEN CHABANE AMEL
- TAFAT MELIA

Réalisé par KabySu

Encadreur: -Nicolas BASKIOTIS
- Olivier SIGAUD

Introduction:

Le Robot doit inspecter l'espace en traçant un carré sauf qu'un obstacle peut tout bloquer...
L'objectif est de lui apprendre à détecter la collision et à s'arrêter directement.
Cela a été implémenté en 2D et 3D.



Robot.py :

```
1  import math
2  import time
3
4  class Robot:
5      def __init__(self, pos_x, pos_y, angle_orientation, vitesse_g, vitesse_d, taille_roue, ecart_roue):
6
7          self.pos_x=pos_x
8          self.pos_y=pos_y
9          self.angle_orientation=angle_orientation
10         self.vitesse_d=vitesse_d
11         self.vitesse_g=vitesse_g
12         self.taille_roue=taille_roue
13         self.ecart_roue=ecart_roue
14         self.rayon=ecart_roue / 2
15         self.distance=0.0
16
17
18
19     def set_vitesses(self, new_vitesse_g, new_vitesse_d):
20
21         self.vitesse_d=new_vitesse_d
22         self.vitesse_g=new_vitesse_g
23         print(f"Mise a jour des vitesses du moteur: G={new_vitesse_g:.2f}m/S, D={new_vitesse_d:.2f} m/s")
24
25
26     def get_distance(self):
27
28         return self.distance
```

```

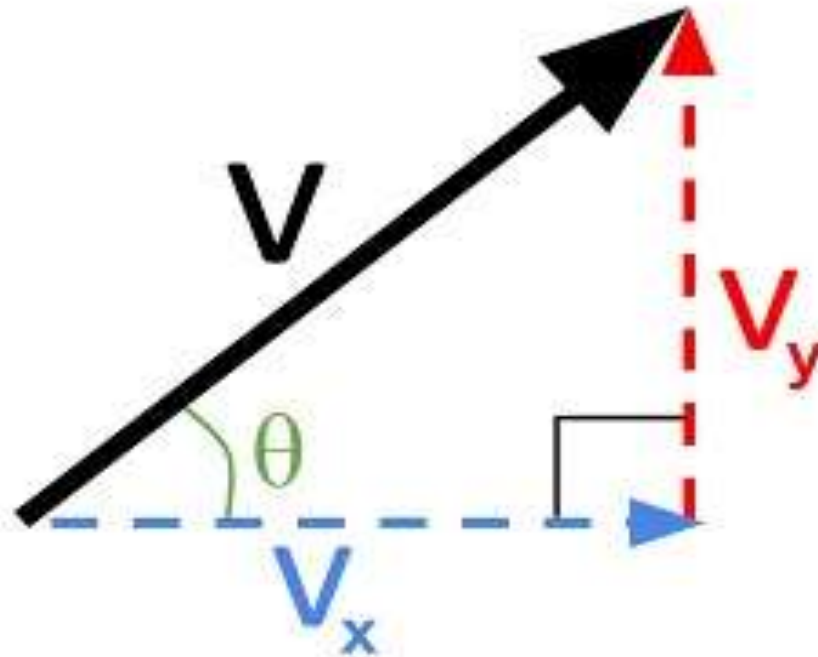
30 def deplacement(self,dt):
31
32     x_prec=self.pos_x
33     y_prec=self.pos_y
34
35     vitesse_moyenne=(self.vitesse_d+self.vitesse_g) / 2
36
37     self.pos_x+=vitesse_moyenne * math.cos(self.angle_orientation) *dt
38     self.pos_y+=vitesse_moyenne * math.sin(self.angle_orientation) * dt
39
40     dx=self.pos_x - x_prec
41     dy=self.pos_y - y_prec
42
43     self.distance+=math.sqrt((dx**2)+(dy**2))
44
45     print(f"Nouvelle coord x :{self.pos_x} et coord y :{self.pos_y} | la distance parcourue est :{self.distance: .3f}m")
46
47
48
49 def rotation(self,dt):
50
51     if self.vitesse_d != self.vitesse_g:
52         delta_angle=(self.vitesse_d - self.vitesse_g)/self.ecart_roue *dt
53         self.angle_orientation+=delta_angle
54         self.angle_orientation %= (2*math.pi) #normalisation entre 0 et 2pi
55         print(f"Rotation : Angle={math.degrees(self.angle_orientation):.1f}")#convertit des angles exprime en radians en degre
56

```

$\text{Pos-x} = V_x \cdot dt$ sachant que $V_x = V_m \cdot \cos \theta$

$\text{Pos-y} = V_y \cdot dt$ sachant que $V_y = V_m \cdot \sin \theta$

C'est ce que nous
avons traduit dans la
méthode déplacement.



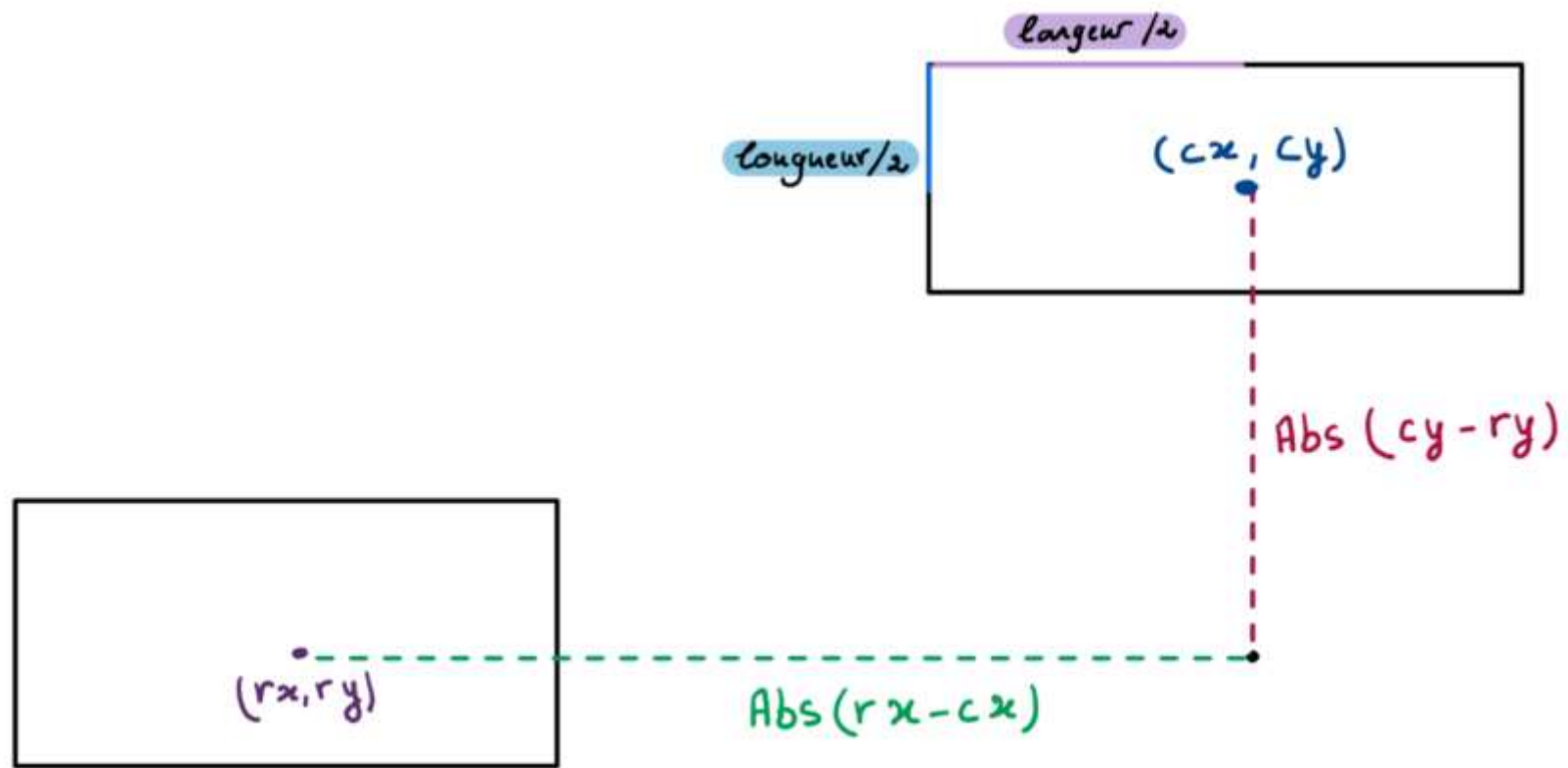
Arene.py :

Classe Obstacle:

```
1  import math
2
3  class Obstacle:
4      def __init__(self,x,y,larg,longueur):
5          self.x=x
6          self.y=y
7          self.larg=larg
8          self.longueur=longueur
9
10     def collision(self,rx,ry,rayon):
11         """detecte si le robot touche un certain obstacle"""
12         cx = self.x + self.larg/2
13         cy = self.y + self.longueur/2
14
15         dx=abs(rx-cx)- self.larg/2
16         dy=abs(ry-cy)- self.longueur/2
17
18         dx2=max(dx,0)
19         dy2=max(dy,0)
20
21         return (dx2 **2 +dy2 **2)<= rayon **2
22
```


Classe arene :

```
23 class Arene:
24     def __init__(self, largeur, hauteur):
25         self.largeur=largeur
26         self.hauteur=hauteur
27         self.liste_obstacles=[]
28         self.robot_actuel=None
29
30
31     def placer_robot(self, robot):
32         self.robot_actuel=robot
33
34     def ajout_obstacles(self, obstacle):
35         self.liste_obstacles.append(obstacle)
36
37     def gerer_collisions(self, robot):
38         """
39         Gère les collisions avec les obstacles.
40         """
41         for obstacle in self.liste_obstacles:
42             collision = obstacle.collusion(robot.pos_x, robot.pos_y, robot.rayon)
43             if collision:
44                 print("Collision détectée!")
45                 robot.set_vitesses(0,0)
46                 break
```



adaptater.py :

```
1  import math
2  import time
3
4  class AdapterVirtuel:
5      def __init__(self, robot, obstacles):
6          self.robot = robot
7          self.obstacles = obstacles
8          self.angle = 0
9          self.derniere_maj= time.time()
10
11         if robot:
12             self.ecart_roue=robot.ecart_roue
13             self.rayon_robot= robot.rayon
14             self.rayon_roue= robot.taille_roue /2
15
16
17         def definir_vitesse(self,v_gauche, v_droite):
18             if self.robot:
19                 self.robot.set_vitesses(v_gauche,v_droite)
20                 self.derniere_maj=time.time()
21
22         def get_dt(self):
23             current_time=time.time()
24             dt=current_time -self.derniere_maj
25             self.derniere_maj=current_time
26             return dt
27
28         def update(self):
29             self.derniere_maj = time.time()
```

```
31 def avancer(self):
32     dt=self.get_dt()
33     self.robot.deplacement(dt)
34     self.update()
35
36 def tourner(self):
37     dt=self.get_dt()
38     self.robot.rotation(dt)
39     self.update()
40
41
42 def dist_parcourue(self):
43     return self.robot.get_distance()
44
45 def get_ang_parcourue(self):
46     dt=self.get_dt()
47     dt_vitesse=self.robot.vitesse_d-self.robot.vitesse_g
48     angle=(dt_vitesse/self.robot.ecart_roue) * dt
49     self.angle+=angle
50     return math.degrees(self.angle)
51
```

controller.py :

Classe controller :

```
1  import math
2  import time
3
4  class Controller:
5      def __init__(self, adapter, arene):
6          self.adapter = adapter
7          self.arene = arene
8
9      def run_simulation(self, strategie, view=None):
10         active = True
11
12         print("Simulation démarrée !")
13         strategie.start(self.adapter)
14
15         while active:
16             termine = strategie.update(self.adapter)
17             if termine:
18                 active = False
19
20             self.adapter.update()
21             self.arene.gerer_collisions(self.adapter.robot)
22             robot = self.adapter.robot
23             print(f"Robot -> x:{robot.pos_x:.2f}, y:{robot.pos_y:.2f}, angle:{robot.angle_orientation:.2f}")
24
25             if view:
26                 view.update_affichage()
27
28             time.sleep(0.1)
29
30         strategie.stop(self.adapter)
31         print("Simulation terminée !")
```

Classe StrategieAvance :

```
33 class StrategieAvance:
34     def __init__(self,distance_cible,vitesse):
35         self.distance_cible=distance_cible
36         self.vitesse=vitesse
37         self.depart=0.0
38
39     def start(self,adapter):
40         self.depart=adapter.dist_parcourue()
41         adapter.definir_vitesse(self.vitesse,self.vitesse)
42
43     def update(self,adapter):
44         adapter.avancer()
45         distance_actuelle=adapter.dist_parcourue()
46         if(distance_actuelle -self.depart)>=self.distance_cible:
47             return True
48         return False
49
50     def stop(self,adapter):
51         adapter.definir_vitesse(0,0)
```

Classe StrategieTournier

```
55 class StrategieTournier:
56     def __init__(self, angle_deg, vitesse_rotation):
57         self.angle_deg = angle_deg
58         self.vitesse = vitesse_rotation
59         self.angle_depart = None
60         self.angle_cible = None
61
62     def start(self, adapter):
63         self.angle_depart = adapter.robot.angle_orientation
64         self.angle_cible = self.angle_depart + math.radians(self.angle_deg)
65
66         if self.angle_deg > 0:
67             adapter.definir_vitesse(-self.vitesse, self.vitesse)
68
69         else:
70             adapter.definir_vitesse(self.vitesse, -self.vitesse)
71
72     def update(self, adapter):
73         adapter.tourner()
74         angle_courant = adapter.robot.angle_orientation
75         difference = self.angle_cible - angle_courant
76
77         if (self.angle_deg > 0 and difference <= 0) or (self.angle_deg < 0 and difference >= 0):
78             adapter.robot.angle_orientation = self.angle_cible
79
80         return True
81     return False
82     def stop(self, adapter):
83         adapter.definir_vitesse(0, 0)
```


Classe StrategieConditionnelle:

```
87 class StrategieConditionnelle:
88     def __init__(self, condition_fonction, action1, action2):
89         self.condition_fonction = condition_fonction
90         self.action1 = action1
91         self.action2 = action2
92         self.action_courante = None
93
94     def start(self, adapter):
95         if self.condition_fonction(adapter):
96             self.action_courante = self.action1
97         else:
98             self.action_courante = self.action2
99         self.action_courante.start(adapter)
100
101     def update(self, adapter):
102         if self.action_courante:
103             fini = self.action_courante.update(adapter)
104             if fini:
105                 self.action_courante.stop(adapter)
106             return True
107         return False
108
109     def stop(self, adapter):
110         if self.action_courante:
111             self.action_courante.stop(adapter)
```


Classe StrategieSequentielle:

```
115 class StrategieSequentielle:
116     def __init__(self, liste_etapes):
117         self.etapes=liste_etapes
118         self.position=0
119
120     def start(self, adapter):
121         if len(self.etapes) >0:
122             self.position=0
123             self.etapes[self.position].start(adapter)
124
125
126     def update(self, adapter):
127         if self.position <len(self.etapes):
128             fini=self.etapes[self.position].update(adapter)
129             if fini:
130                 self.etapes[self.position].stop(adapter)
131                 self.position+=1
132                 if self.position <len(self.etapes):
133                     self.etapes[self.position].start(adapter)
134
135
136
137         return self.position >=len(self.etapes)
138
139
140     def stop(self, adapter):
141         if self.position <len(self.etapes):
142             self.etapes[self.position].stop(adapter)
```

View_2D.py :

Classe View :

```
1  import tkinter as tk
2  import math
3
4  class View(tk.Tk):
5      def __init__(self, arene, robot):
6          super().__init__()
7          self.title("Simulation Robot")
8          self.arene = arene
9          self.robot = robot
10         self.canvas = tk.Canvas(self, width=arene.largueur, height=arene.hauteur, bg="white")
11         self.canvas.pack()
12         self.trace_points = []
13
14         def update_affichage(self):
15             self.canvas.delete("all")
16
17             # Affichage de la bordure
18             margin = 10
19             self.canvas.create_rectangle(
20                 margin, margin,
21                 self.arene.largueur - margin,
22                 self.arene.hauteur - margin,
23                 outline="black"
24             )
25
26             # Obstacles
27             for obs in self.arene.liste_obstacles:
28                 self.canvas.create_rectangle(obs.x, obs.y, obs.x + obs.larg, obs.y + obs.longueur, fill="red")
```

```
29
30 # Tracer la trajectoire
31 self.trace_points.append((self.robot.pos_x, self.robot.pos_y))
32 if len(self.trace_points) > 1:
33     coords = []
34     for (x, y) in self.trace_points:
35         coords.extend([x, y])
36     self.canvas.create_line(*coords, fill="green", width=2)
37
38 # Corps du robot en bleu
39 L = self.robot.ecart_roue
40 l = L + self.robot.taille_roue
41 half_L = L / 2
42 half_l = l / 2
43 theta = self.robot.angle_orientation
44 cos_t = math.cos(theta)
45 sin_t = math.sin(theta)
46
47 # Coins du rectangle du robot
48 points_local = [
49     ( half_L, -half_l),
50     ( half_L,  half_l),
51     (-half_L,  half_l),
52     (-half_L, -half_l)
53 ]
54 poly_points = []
55 for (x_local, y_local) in points_local:
```

```
56     x_global = self.robot.pos_x + x_local * cos_t - y_local * sin_t
57     y_global = self.robot.pos_y + x_local * sin_t + y_local * cos_t
58     poly_points.extend([x_global, y_global])
59
60     self.canvas.create_polygon(poly_points, fill="blue")
61
62     # Roues
63     vx = math.cos(theta)
64     vy = math.sin(theta)
65     px = -vy
66     py = vx
67     x_center = self.robot.pos_x
68     y_center = self.robot.pos_y
69     half_dist = self.robot.ecart_roue / 2
70     r = self.robot.taille_roue / 2
71
72     # Roue gauche
73     x_left = x_center + px * (-half_dist)
74     y_left = y_center + py * (-half_dist)
75     self.canvas.create_oval(x_left - r, y_left - r, x_left + r, y_left + r, fill="black")
76
77     # Roue droite
78     x_right = x_center + px * (half_dist)
79     y_right = y_center + py * (half_dist)
80     self.canvas.create_oval(x_right - r, y_right - r, x_right + r, y_right + r, fill="black")
81
82     self.update()
```


View_3D.py :

Classe View3D :

```
1  from ursina import *
2  import math
3
4  hauteur_o=10
5  hauteur_r=2
6
7  class View3D:
8      def __init__(self, arene, robot):
9          #initialiser l'application ursina
10         self.app=Ursina()
11         self.arene=arene
12         self.robot=robot
13
14         window.title="Simulation 3D Robot"
15         window.borderless=False
16         window.fullscreen=False
17         window.exit_button.visible=True
18         window.fps_compteur=True #activer l'affichage du nombre de fps
19         window.color=color.gray
20
21         largeur=arene.largeur
22         hauteur=arene.hauteur
23
24         self.sol=Entity(model='plane', texture='white_cube', scale=(largeur,1,hauteur), position=(largeur/2,0,hauteur/2), texture_scale=(largeur,hauteur))
25         self.robot_ent=Entity(model='cube', color=color.rgb(0,120,255,100), scale=(15,6,15), position=(robot.pos_x, hauteur_r/2, robot.pos_y))
26
27         self.obstacle_entities = []
28
29         for obs in arene.liste_obstacles:
```



```

29     for obs in arene.liste_obstacles:
30         obj=Entity(model='cube',
31                     color=color.pink,
32                     scale=(obs.larg,hauteur_o,obs.longueur),
33                     position=(obs.x+obs.larg/2,hauteur_o/2,obs.y+obs.longueur/2))
34         self.obstacle_entities.append(obj)
35
36     self.camera=EditorCamera(position=(largeur/2,350,hauteur/2-410),rotation=(45,0,0))
37     camera.fov=70
38
39     #pour cacher les elements d'interface par default d'Ursina
40     camera.ui.enabled=False
41
42     self.label=Text(text='',origin=(0,18),background=True)
43
44     class Input_handler(Entity):
45         def __init__(self,parent_view):
46             super().__init__()
47             self.parent_view=parent_view
48
49             def input(self,key):
50                 if key=='escape':
51                     application.quit()
52
53
54
55     self.input_handler=Input_handler(self)
56
57     def update_affichage(self):
58         self.robot_ent.position=(self.robot.pos_x,hauteur_r/2,self.robot.pos_y)
59         self.robot_ent.rotation_z=-math.degrees(self.robot.angle_orientation)#ursina tourne par default dans le sens horaire
60
61         self.label.text=f"Pos:({self.robot.pos_x :.1f},{self.robot.pos_y : .1f})\n"+ \
62             f"Vg :{self.robot.vitesse_g :.1f}, Vd:{self.robot.vitesse_d :.1f}"
63
64     self.app.step() #pour mettre a jour la scene

```

Conclusion :

**On vous remercie pour
votre attention.**